

Chapitre II L'apprentissage supervisé : cas de la régression

1. Introduction

C'est la technique d'apprentissage la plus utilisée. Cette technique est une phase de réglage des paramètres du modèle proposé. Cette phase consiste à présenter un ensemble de données formées par une ou des caractéristiques d'entrées et une ou des caractéristiques de sorties. Ces couples Entrées-Sorties seront liés par des relations mathématiques qui définissent le modèle. L'apprentissage supervisé est défini par **4 notions fondamentales** : la notion de, la notion duutilisé, la notion de la (appelé aussi fonction) et la notion de Ces notions seront expliquées dans le cas de la régression (dans ce chapitre) et dans le cas de la classification (chapitre 3).

2. Structure de la base de données

La base de donnée appelée aussi 'data set' est un tableau de données formé par un ou plusieurs facteurs appelé 'features' noté généralement par x_i et une ou plusieurs sorties appelées 'Target' généralement noté par y_i . La base de donnée est composée par un ensemble d'exemples de même dimension, de même nombre de facteur d'entrées et de même caractéristiques de sorties. Le tableau suivant présente un exemple de 'data set' dans lequel on définit le prix d'une voiture en fonction de plusieurs caractéristiques.

Tableau 2.1 : Présentation de différents exemples spécifique aux prix de la voiture.

Entrées			Sortie
Age (x_1)	Puissance(x_2)	Kilométrage(x_3)	Prix
60	4	110	22
48	7	90	31
12	4	20	45
30	5	45	43

Par convention le nombre d'exemples est noté m et n_E est le nombre d'entrées ou le nombre des features. Le nombre de sortie est noté par n_s . On note la valeur du 2^{ème} exemple de la 3^{ème} entrée par : $x_3^{(2)} =$.

On parle alors de la matrice de la base entière suivante :

$$\begin{bmatrix} & & & & \end{bmatrix}$$

On peut aussi écrire :

Le vecteur de sortie $Y = \begin{bmatrix} \\ \\ \end{bmatrix}$ avec $\dim(Y)=[\dots\dots\dots]$ (ou dans le cas générale $\dim(y)=[\dots\dots\dots]$)

La matrice d'entrée s'écrit : $X = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$ avec $\dim(X)=[\dots\dots\dots]$

3. Modèle

3.1 Représentation des données

Il suffit d'avoir la base de donnée, on peut représenter la sortie en fonction d'une ou des sorties. Cette représentation devient plus délicate lorsque le nombre d'entrées devient supérieur à 3. Des exemples de représentation des données sont illustrés ci-dessous selon le nombre d'entrées.

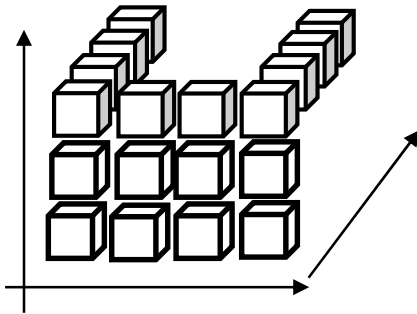
- Dans le cas de deux entrées :

- Cas de trois entrées

- Cas de 4 entrées

- Cas de 5 entrées

Cas de 6 entrées



Remarque

La représentation de la sortie en fonction des entrées devient de plus en plus complexe et difficile avec l'augmentation du nombre d'entrées.

3.2 Modèle à une seule variable d'entrée

Tenant compte du cas de l'exemple illustré dans le tableau précédent, la variation de la valeur de la voiture y en fonction du nombre de kilométrage est représentée dans la figure suivante.

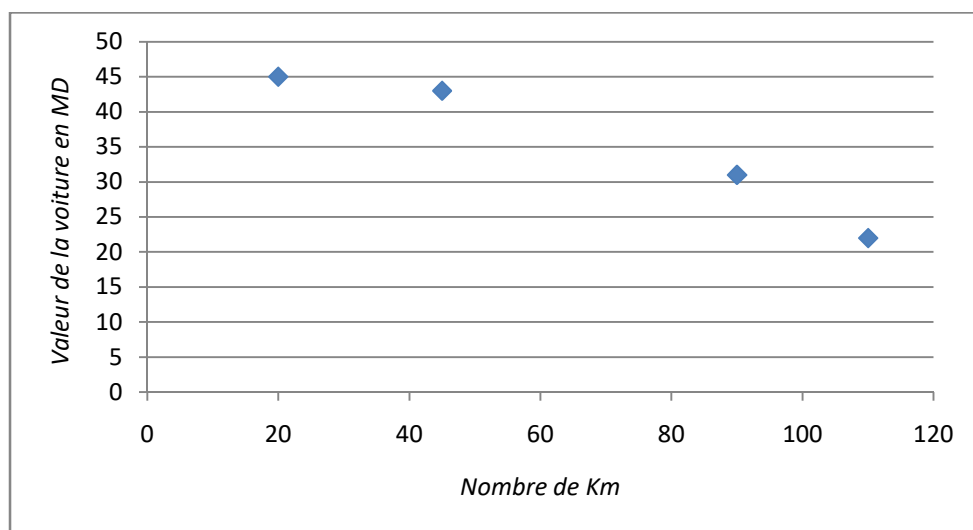


Figure 2.1 : Variation du prix de la voiture en fonction du nombre de kilométrage.

On peut choisir les modèles mathématiques suivants :

- **Modèle linéaire :**

.....

On appelle aussi la forme d'un modèle de régression linéaire.

- **Modèle polynomial d'ordre 2**

$$\hat{y} =$$

- **Modèle polynomial d'ordre 3**

$$\hat{y} =$$

Avec (a,b,c,d) sont les paramètres du modèle à déterminer et c'est l'algorithme d'apprentissage supervisé qui va nous permettre de calculer et de trouver ces paramètres à partir de la base de données. La figure suivante montre une approximation des 4 points de la base de données par trois modèles différents : un modèle linéaire, un modèle d'ordre 2 et un modèle d'ordre 3. Leurs équations sont notées sur la figure.

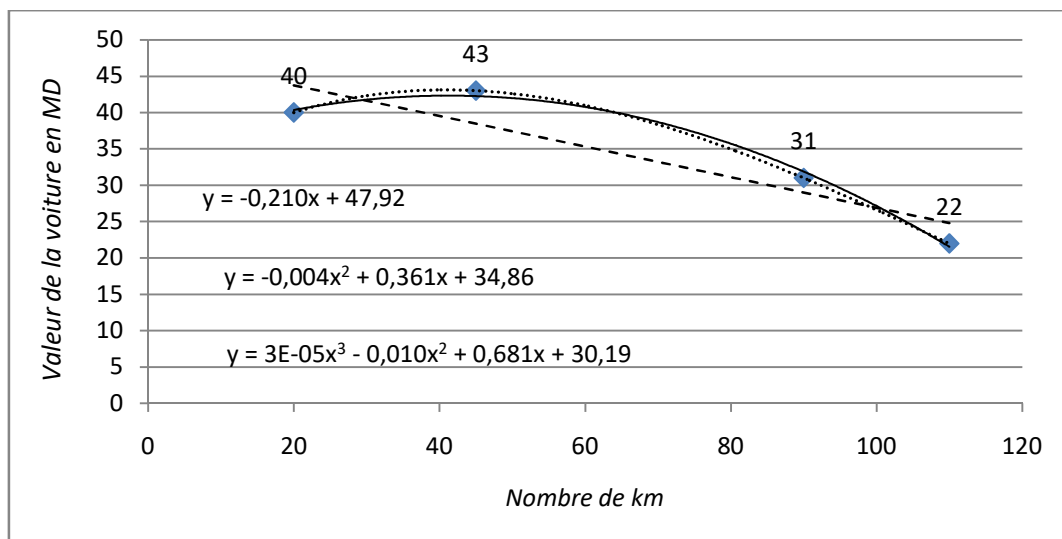


Figure 2.2 : Approximation des trois modèles de la variation de la valeur de la voiture.

3.3 Modèle à multi variables d'entrées

Si on s'intéresse à la variation de y en fonction de plusieurs caractéristiques (ou features) tel que :

$$\hat{y} = f(x_1, x_2, x_3)$$

- **Modèle linéaire :**

L'expression du modèle linéaire devient sous la forme suivante :

$$\hat{y} =$$

Dans ce cas on a paramètres à trouver au lieu deparamètres dans le cas d'un seul attribut.

- **Modèle polynomiale d'ordre 2**

L'expression du modèle polynomial d'ordre 2 des trois entrées s'écrit :

$$\hat{y} =$$

Remarque

(a, b, c, d) ne sont pas les mêmes paramètres dans le modèle linéaire et le modèle d'ordre 2. C'est pour cette raison la notation des paramètres sera notée par a_i . Ainsi le modèle d'ordre 2 devient :

$$\hat{y} =$$

L'évolution des modèles vers un ordre supérieur permet d'avoir de bonnes performances de prédiction de la base d'apprentissage. En contrepartie le nombre de paramètres explose et devient important. Ceci favorise le phénomène de sur apprentissage du modèle.

La figure suivante montre le résultat du choix du niveau de complexité du modèle sur les performances de prédiction d'apprentissage et les performances de prédiction dans la généralisation.

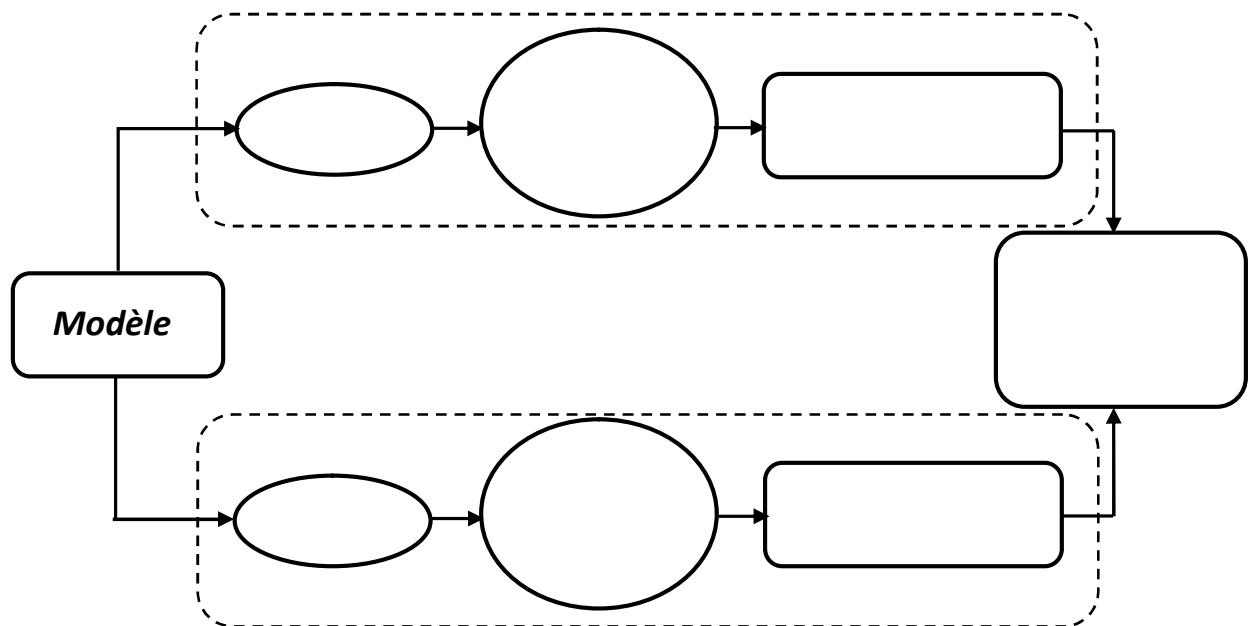


Figure 2.3 : Effet de la complexité du modèle sur les performances d'apprentissage et de généralisation

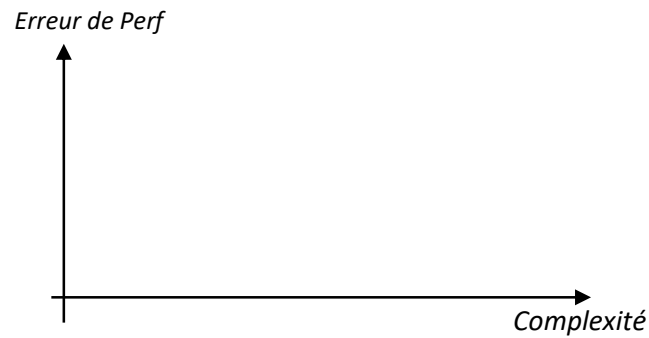


Figure 2.4 : Allure de la variation des performances d'apprentissage et de généralisation en fonction de la complexité du modèle

4. Fonction coût

C'est le critère qui permet de choisir si le modèle est performant ou non. Ce critère s'écrit généralement en fonction de l'écart entre la valeur produite du modèle notée par $\hat{y}$ et la valeur réelle contenue dans la base notée par y qui s'écrit sous la forme suivante :

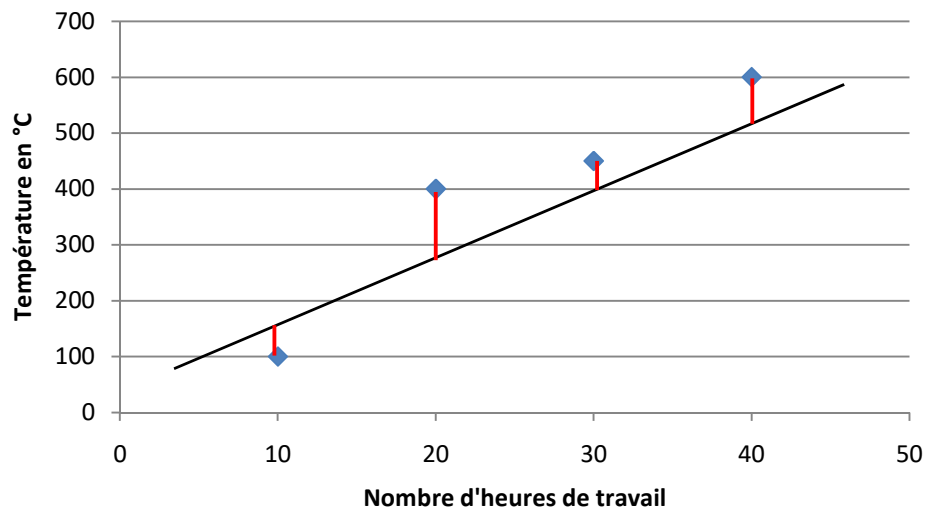


Figure 2.5 : Représentation graphique de l'erreur du modèle par rapport aux valeurs réelles

Le modèle présente de bonnes performances si ces erreurs sont nulles ce qui n'est pas toujours le cas. D'une façon générale, on peut évaluer l'erreur globale tenant compte de m exemples dans la base comme suit :

$$e =$$

Equivalent à :

$$e =$$

D'une manière **efficace** et pour éviter le signe négatif, on utilise le carré de l'erreur. Cette forme s'appelle l'erreur quadratique moyenne notée par : $mse(MSE : mean square error)$. Dans ce cas, la fonction du coût et notée par J et s'écrit :

$$J =$$

Cette erreur est en fonction de la moyenne du carré de la distance entre y_i et $\hat{y}_i$. Si cette distance est minime, elle favorise la sélection du bon modèle. C'est pour cela qu'il faut déterminer une stratégie qui permet de trouver les paramètres du modèle (a, b, c, d) qui tend à minimiser la fonction coût. Cette stratégie est nommée « algorithme d'apprentissage ».

5. Algorithme d'apprentissage

L'algorithme d'apprentissage est une stratégie qui permet de trouver les paramètres du modèle (a_i) qui permettent d'avoir la valeur minimale de la fonction coût totale. L'algorithme d'apprentissage le plus connu est l'algorithme de la descente du gradient. Cet algorithme a montré une flexibilité et des performances intéressantes dans la majorité des applications et par rapport aux algorithmes classiques comme l'algorithme des moindres carrés.

1. Cas d'une régression linéaire à une seule variable :

Dans ce cas, la forme du modèle est :

$$\hat{y} =$$

Ce modèle dépend que de a et b ce qui implique que la fonction coût J dépend aussi de a et de b qui s'écrit :

$$J =$$

On peut écrire aussi :

$$J(a, b) =$$

$J(a, b)$ s'écrit en fonction d'une somme carrée de a ou b . On peut alors la représenter selon la figure suivante :



Figure 2.6 : Allure de la variation de l'erreur en fonction du paramètre a

Pour atteindre la valeur minimale de J , les méthodes classiques utilisées sont : la méthode des moindres carrés et la méthode de la descente du gradient. Actuellement, la méthode des moindres carrés n'est plus utilisée à cause du temps de calcul nécessaire lors de l'utilisation des bases de données importantes (ce qui est le cas de la majorité des applications). Tandis que la méthode du gradient a donné des bonnes performances en termes de temps de calcul et en terme d'erreur d'apprentissage.

Pour calculer la valeur optimale de a_{op} , on a recours au calcul de l'équation de la tangente à la courbe en ce point.

Généralement l'équation d'une tangente en un point donné (d'abscisse a) s'écrit :

$$y = f(a) + f'(a)(x - a)$$

$$y = f'(a) * x + f(a) - f'(a)a$$

Ainsi une tangente en un point d'abscisse a possède une pente égale à $f'(a)$.

Dans ce cas on n'a pas la fonction $f(a)$ est remplacée par $J(a, b)$. Par conséquent, l'équation de la tangente s'écrit :

$$y = J'(a) * x + J(a) - J'(a)a$$

Avec $J'(a) = \frac{\partial J}{\partial a}$

Donc l'équation de la pente de la tangente s'écrit en fonction de : $J'(a)$ ou $\frac{\partial J}{\partial a}$

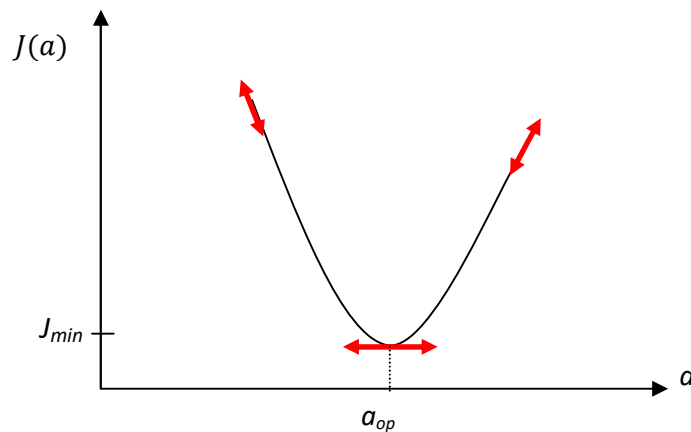


Figure 2.7 : Exemple de variation de l'erreur en fonction de son paramètre

La méthode du gradient consiste à :

1. Choisir un point aléatoire (choisir une valeur aléatoire de a),
2. Calculer : $\frac{\partial J}{\partial a}$,
3. Faire un pas en fonction de **l'opposé de la dérivée** (c'est-à-dire en fonction de : $-\frac{\partial J}{\partial a}$) jusqu'à **atteindre le changement** de signe de : $\frac{\partial J}{\partial a}$.
4. Ce changement de signe montre que l'algorithme atteint le minimum de la courbe J_{min} .

La formulation de l'étape 3 s'écrit alors comme suit :

$$a_{i+1} = a_i - \alpha \frac{\partial J}{\partial a}$$

Avec

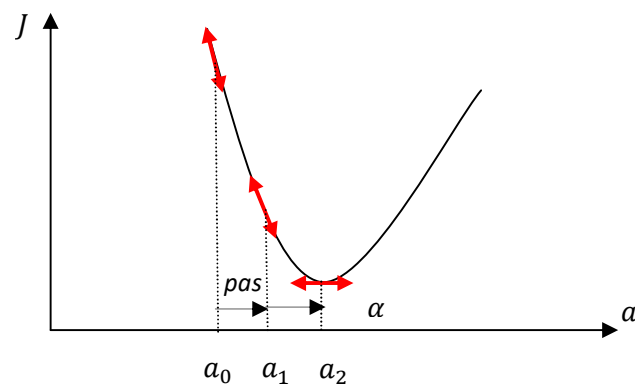
a_{i+1} est la nouvelle valeur,

a_i est l'ancienne valeur,

α : est un hyper paramètre appelé le pas d'apprentissage (Learning rate toujours >0).

Remarque

- Dans le cas où α est de valeur importante on aura une oscillation de l'algorithme.
- Dans le cas où α est faible et proche de 0, la descente du gradient est lente et provoque un temps d'apprentissage important.



On applique le même raisonnement pour les autres paramètres du modèle. Dans le cas d'une régression linéaire avec une seule entrée ($y = ax + b$), on a deux paramètres a et b leurs formulations s'écrivent :

$$a_{i+1} = a_i - \alpha \frac{\partial J}{\partial a}$$

$$b_{i+1} = b_i - \alpha \frac{\partial J}{\partial b}$$

Il faut donc, calculer le gradient $\frac{\partial J}{\partial a}$ et $\frac{\partial J}{\partial b}$ afin de trouver les nouvelles valeurs respectives de a_{i+1} et b_{i+1} . Sachant que l'équation de base :

$$J(a, b) = \quad =$$

Donc :

$$\frac{\partial J}{\partial a} = \frac{1}{m} \sum_{i=1}^m x_i (ax_i + b - y_i)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m 1 (ax_i + b - y_i)$$

Rappel mathématique

$$(g \circ f)' = f' * g'(f)$$

Dans ce cas :

$$g(f) = f^2 \text{ et } f = ax + b - y$$

6. Ecriture matricielle des différentes formulations de l'algorithme du gradient descendant

Dans cette partie, les différentes équations du modèle de la fonction cout et de l'algorithme du gradient descendant seront présentées sous forme matricielle. Cette forme permet de simplifier les calculs, de développer des modèles plus complexes sans changer la nature des opérations entre matrices et de préparer toute la procédure à l'insertion à la programmation que ce soit sur : Matlab, Python, R etc..

1. Cas de la régression linéaire

- **Base de données**

On note généralement la matrice de la base complète des données par B qui s'écrit :

$$B = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_{n_E}^{(1)} & y^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_{n_E}^{(m)} & y^{(m)} \end{bmatrix}$$

On peut extraire la matrice des entrées X_b et la matrice de sortie Y tel que :

$$X_b = \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \quad \text{et} \quad Y = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

- **Modèle** : Dans ce cas pour faciliter la compréhension on peut utiliser un modèle linéaire avec une seule entrées ou un modèle linéaire avec 3 entrées tel que :

Modèle 1 : $y =$

Modèle 2 : $y =$

On peut écrire alors

$$Y_{(y_i)} = X_{(x_i)} \cdot \theta_{(a,b)}$$

Dans le cas du modèle 1

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix} \cdot \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

L'équation aux dimensions s'écrit :

Dans le cas du modèle 2

$$X = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \quad \text{et} \quad \theta = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

L'équation devient :

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} = \begin{bmatrix} x \\ x \\ \vdots \\ x \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

On peut conclure que l'écriture matricielle du modèle est :

$$Y = X\theta$$

L'équation aux dimensions s'écrit :

$$[m \times 1] = [m \times 4] \times [4 \times 1]$$

D'une façon générale cette équation devient :

$$[m \times 1] = [m \times n] \times [n \times 1]$$

- **Fonction coût :**

Il faut savoir que la fonction coût est une valeur (en d'autres termes de taille [1x1]) qu'on note par J ou $J(a, b)$ et qui s'écrit :

$$J(a, b) = J(\theta) = \frac{1}{2m} \sum_{i=1}^m (ax_i + b - y_i)^2$$

L'écriture matricielle de la fonction coût devient alors :

$$J(\theta) = \frac{1}{2m} (X\theta - Y_B)^2$$

Avec :

$X\theta$ est de dimension $[m, 1]$

Y_B est de dimension $[m, 1]$

donc $X\theta - Y_B$ est de dimension $[m, 1]$

de même $(X\theta - Y_B)^2$ est de dimension $[m, 1]$

- **Gradient descendant :**

D'après les équations trouvées, les équations du gradient sont :

$$\frac{\partial J(a, b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x(ax + b - Y_{B_i})$$

$$\frac{\partial J(a, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m 1(ax + b - Y_{B_i})$$

Puisque

$$\sum_{i=1}^m ax + b - Y_{B_i} = X\theta - Y_B$$

On peut écrire alors

L'équation matricielle du gradient descendant est

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T (X\theta - Y_B)$$

Avec

$$X = \begin{bmatrix} x_1^{(1)} & 1 \\ x_1^{(2)} & 1 \\ \vdots & \vdots \\ x_1^{(m)} & 1 \end{bmatrix} \text{ et } X^T = \begin{bmatrix} & \\ & \end{bmatrix}$$

Puisque $X\theta - Y_B$ est de dimension $[m, 1]$

$X^T (X\theta - Y_B)$ est de dimension $[2, m] [m, 1] = [2, 1]$

Et dans le cas général :

$X^T (X\theta - Y_B)$ est de dimension $[N_E + 1, 1]$

Cette formule est valable pour calculer tous les gradients et par suite on peut exprimer les nouvelles valeurs de paramètres du modèle (a, b) (ou $(a, b, c, \dots)$ ou θ dans le cas général) à travers les expressions suivantes :

$$\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a - \alpha \frac{\partial J}{\partial a} \\ b - \alpha \frac{\partial J}{\partial b} \end{bmatrix}$$

Dans le cas général l'équation des paramètres est

$$\theta =$$

Equations aux dimensions :

$$\begin{aligned} [(N_E + 1) * 1] &= [(N_E + 1) * 1] - [1 * 1][(N_E + 1) * 1] \\ &= [(N_E + 1) * 1] - [(N_E + 1) * 1] \\ &= [(N_E + 1) * 1] \end{aligned}$$

L'équation matricielle de l'ajustement des paramètres du modèle est :

$$\theta = \theta - \alpha \frac{1}{m} X^T (X\theta - Y_B)$$

2. Cas de la régression non linéaire

Dans le cas de la régression non linéaire tel que :

$$y =$$

- **Base de données :**

La base de données reste la même.

- **Le modèle :**

Le modèle s'écrit comme suit :

$$Y = \quad \text{(dans le cas d'une seule variable)}$$

$$Y = \quad \text{(dans le cas de plusieurs variables)}$$

Y S'écrit toujours sous la forme suivante :

$$Y = X\theta$$

Avec :

$$X = \begin{bmatrix} \\ \\ \end{bmatrix} \text{ et } \theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

L'équation aux dimensions s'écrit comme suit :

Dans le cas du 2^{ème} modèle polynomial d'ordre 2 à trois variables, l'équation matricielle reste la même :

$$Y = X\theta$$

Avec

$$X = \begin{bmatrix} & & & & & \\ & & & & & \\ & & & & & \end{bmatrix} \quad \theta = \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ \vdots \\ \vdots \\ g \end{bmatrix}$$

Et l'équation aux dimensions devient :

Remarque :

Les dimensions de X et de θ changent en fonction du modèle utilisé mais **l'équation matricielle du modèle reste la même.**

Les équations de la fonction coût, du gradient descendant ainsi que l'algorithme de descente du gradient restent aussi inchangées.

7. Récapitulation des étapes du développement d'un modèle de régression :

1- Modèle :

2- Fonction coût :

3- Gradient :

4- Algorithme du gradient ou l'ajustement des paramètres :